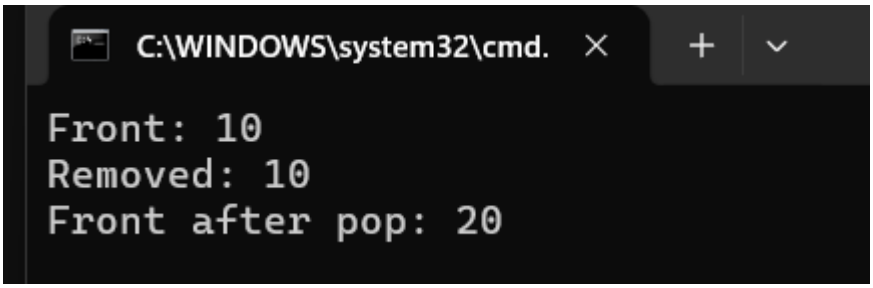
DAY 6:

HACKERRANK:

1. Queue using Two Stacks
   CODE:
```
#include <iostream>
#include <stack>
using namespace std;

class MyQueue {
    stack<int> s1, s2;

public:
    void push(int x) {
        s1.push(x);
    }

    int pop() {
        if (s2.empty()) {
            while (!s1.empty()) {
                s2.push(s1.top());
                s1.pop();
            }
        }
        int val = s2.top();
        s2.pop();
        return val;
    }

    int peek() {
        if (s2.empty()) {
            while (!s1.empty()) {
                s2.push(s1.top());
                s1.pop();
            }
        }
        return s2.top();
    }

    bool empty() {
        return s1.empty() && s2.empty();
    }
};

int main() {
    MyQueue q;
```

```
    q.push(10);
    q.push(20);
    q.push(30);

    cout << "Front: " << q.peek() << endl;

    cout << "Removed: " << q.pop() << endl;

    cout << "Front after pop: " << q.peek() << endl;

    return 0;
}
```

OUTPUT:



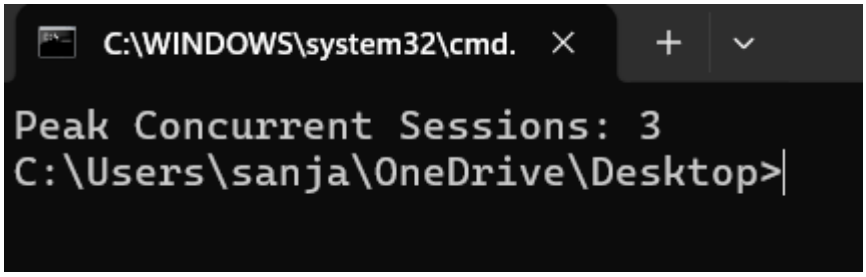
2. Peak Concurrent Sessions (per user node)

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int peakSessions(vector<pair<int,int>> &sessions) {
    vector<pair<int,int>> events;

    for(auto s : sessions){
        events.push_back({s.first,1});
        events.push_back({s.second,-1});
    }

    sort(events.begin(), events.end());

    int curr = 0, peak = 0;

    for(auto e : events){
        curr += e.second;
        peak = max(peak, curr);
    }

    return peak;
}
```

```
int main(){

    vector<pair<int,int>> sessions = {
        {1,5},
        {2,6},
        {4,7},
        {5,8}
    };

    cout<<"Peak Concurrent Sessions: "<<peakSessions(sessions);

    return 0;
}
```

OUTPUT:



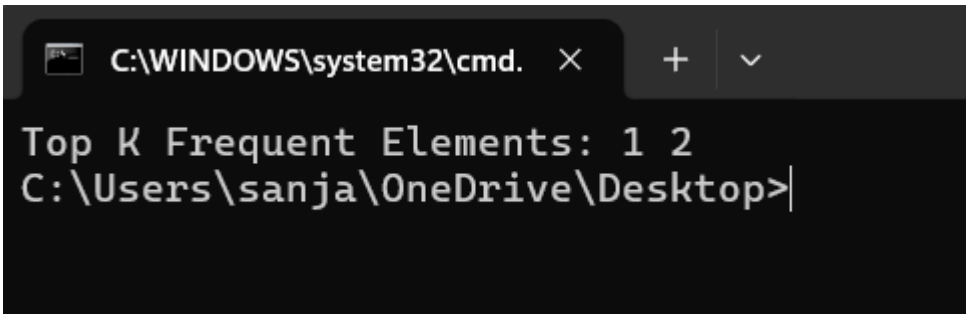
3. Top K Frequent Events

CODE:

```
#include <bits/stdc++.h>
using namespace std;

vector<int> topKFrequent(vector<int>& nums, int k) {

    unordered_map<int,int> freq;

    for(int n : nums)
        freq[n]++;

    vector<pair<int,int>> arr;

    for(auto it : freq)
        arr.push_back({it.second, it.first});

    sort(arr.rbegin(), arr.rend());

    vector<int> res;

    for(int i=0;i<k;i++)
        res.push_back(arr[i].second);
```

```
    return res;
}

int main(){

    vector<int> nums = {1,1,1,2,2,3};
    int k = 2;

    vector<int> result = topKFrequent(nums,k);

    cout<<"Top K Frequent Elements: ";

    for(int x : result)
        cout<<x<<" ";

    return 0;
}
```

OUTPUT:



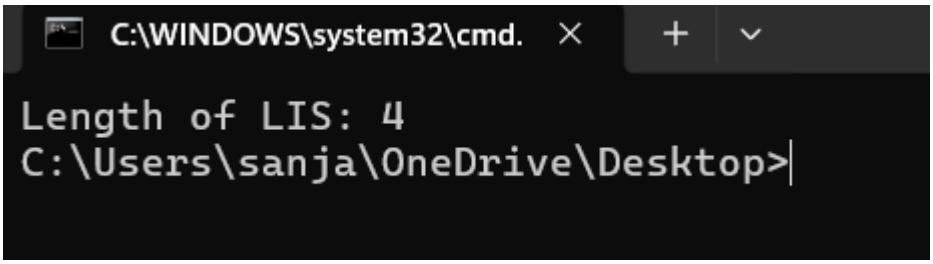
4. Longest Increasing Subsequence Length

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int lengthOfLIS(vector<int>& nums) {

    vector<int> dp;

    for(int n : nums){

        auto it = lower_bound(dp.begin(), dp.end(), n);

        if(it == dp.end())
            dp.push_back(n);
        else
            *it = n;
    }

    return dp.size();
```

```
}

int main(){

    vector<int> nums = {10,9,2,5,3,7,101,18};

    cout<<"Length of LIS: "<<lengthOfLIS(nums);

    return 0;
}
```

OUTPUT:



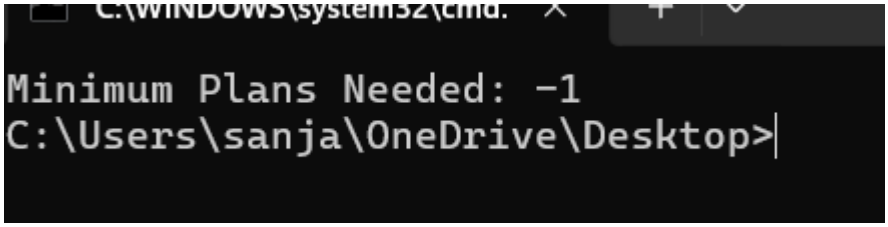
5. Minimum Plans to Reach Target Bandwidth

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int minPlans(vector<int>& plans, int target) {

    vector<int> dp(target+1, INT_MAX);

    dp[0] = 0;

    for(int p : plans){

        for(int i = target; i >= p; i--){

            if(dp[i-p] != INT_MAX)
                dp[i] = min(dp[i], dp[i-p] + 1);
        }
    }

    return dp[target] == INT_MAX ? -1 : dp[target];
}

int main(){

    vector<int> plans = {1,3,5};
    int target = 7;
```

```
    cout<<"Minimum Plans Needed: "<<minPlans(plans,target);

    return 0;
}
```

OUTPUT:



LEETCODE:

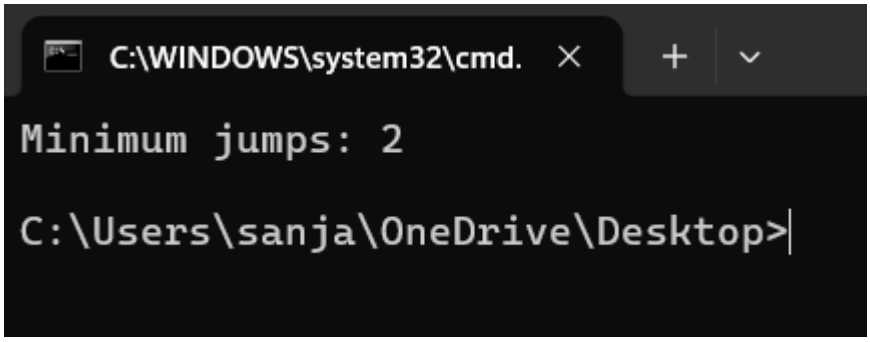
1. Jump Game II

CODE:

```
#include <bits/stdc++.h>

using namespace std;


int jump(vector<int>& nums) {

    int jumps = 0, currEnd = 0, farthest = 0;


    for (int i = 0; i < nums.size() - 1; i++) {

        farthest = max(farthest, i + nums[i]);


        if (i == currEnd) {

            jumps++;

            currEnd = farthest;

        }

    }


    return jumps;

}


int main() {

    vector<int> nums = {2, 3, 1, 1, 4};
```

```
    cout << "Minimum jumps: " << jump(nums) << endl;


    return 0;
}
```

OUTPUT:



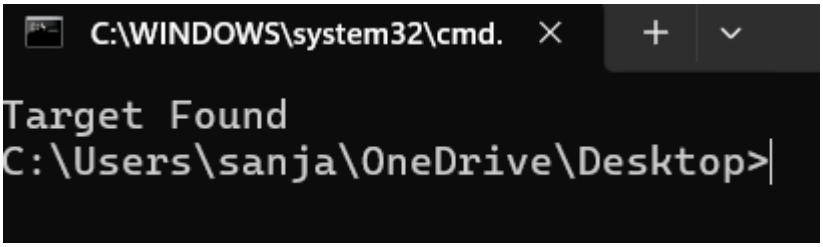
2. Search in Rotated Sorted Array II

   CODE:

```
#include <bits/stdc++.h>
using namespace std;

bool search(vector<int>& nums, int target) {

    int l = 0, r = nums.size()-1;

    while(l <= r){

        int mid = (l+r)/2;

        if(nums[mid] == target)
            return true;

        if(nums[l] == nums[mid] && nums[mid] == nums[r]){
            l++;
            r--;
        }

        else if(nums[l] <= nums[mid]){

            if(nums[l] <= target && target < nums[mid])
                r = mid-1;
            else
                l = mid+1;
```

```
        }

        else{

            if(nums[mid] < target && target <= nums[r])
                l = mid+1;
            else
                r = mid-1;
        }
    }

    return false;
}

int main(){

    vector<int> nums = {2,5,6,0,0,1,2};
    int target = 0;

    if(search(nums,target))
        cout<<"Target Found";
    else
        cout<<"Target Not Found";

    return 0;
}
```

OUTPUT:



3. Recover Binary Search Tree

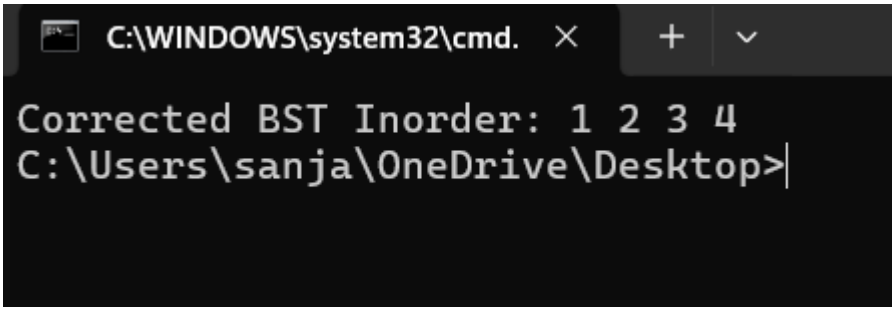
CODE:

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode{
    int val;
    TreeNode *left, *right;

    TreeNode(int x){
        val = x;
        left = right = NULL;
```

```
    }
};

TreeNode *first=NULL,*second=NULL,*prevNode=NULL;

void inorder(TreeNode* root){

    if(!root) return;

    inorder(root->left);

    if(prevNode && root->val < prevNode->val){

        if(!first)
            first = prevNode;

        second = root;
    }

    prevNode = root;

    inorder(root->right);
}

void recoverTree(TreeNode* root){

    inorder(root);

    swap(first->val, second->val);
}

void printInorder(TreeNode* root){
    if(!root) return;

    printInorder(root->left);
    cout<<root->val<<" ";
    printInorder(root->right);
}

int main(){

    TreeNode* root = new TreeNode(3);
    root->left = new TreeNode(1);
    root->right = new TreeNode(4);
    root->right->left = new TreeNode(2);
```

```
    recoverTree(root);

    cout<<"Corrected BST Inorder: ";
    printInorder(root);

    return 0;
}
```

OUTPUT:



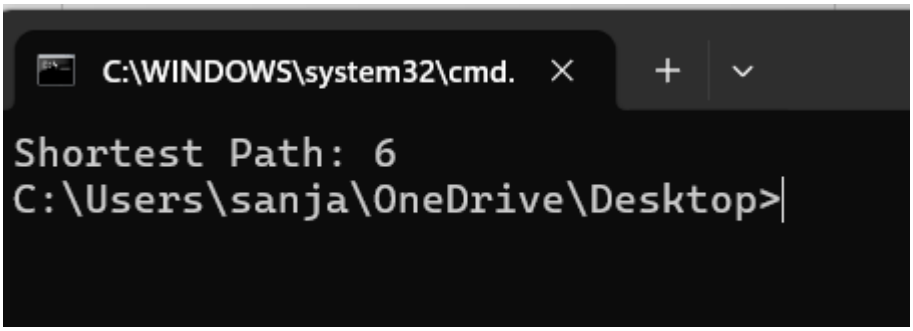
4. Shortest Path with Processing Delay (Dijkstra)

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int shortestPath(int n, vector<vector<pair<int,int>>> &graph, int src, int dest){

    vector<int> dist(n, INT_MAX);

    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;

    dist[src] = 0;

    pq.push({0, src});

    while(!pq.empty()){

        auto top = pq.top();
        int d = top.first;
        int u = top.second;
        pq.pop();

        for(auto edge : graph[u]){

            int v = edge.first;
            int w = edge.second;

            if(dist[v] > d + w){

                dist[v] = d + w;
```

```
            pq.push({dist[v], v});
        }
    }
}

    return dist[dest];
}

int main(){

    int n = 5;

    vector<vector<pair<int,int>>> graph(n);

    graph[0].push_back({1,2});
    graph[0].push_back({2,4});
    graph[1].push_back({2,1});
    graph[1].push_back({3,7});
    graph[2].push_back({4,3});
    graph[3].push_back({4,1});

    cout<<"Shortest Path: "<<shortestPath(n,graph,0,4);

    return 0;
}
```

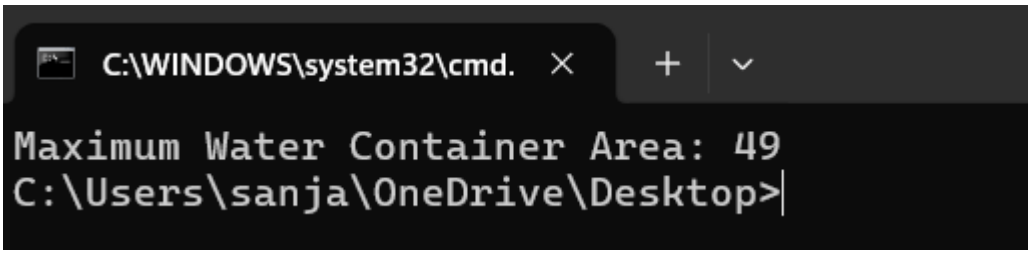
OUTPUT:



5. Container With Most Water

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int maxArea(vector<int>& height) {

    int l = 0, r = height.size()-1;
    int ans = 0;
```

```
    while(l < r){

        int area = min(height[l], height[r]) * (r-l);

        ans = max(ans, area);

        if(height[l] < height[r])
            l++;
        else
            r--;
    }

    return ans;
}

int main(){

    vector<int> height = {1,8,6,2,5,4,8,3,7};

    cout<<"Maximum Water Container Area: "<<maxArea(height);

    return 0;
}
```

OUTPUT: